

Chapter 10 --- DQL: Retrieving and Filtering Data

Chapter 3 introduced DQL as the category used to retrieve information.

This chapter goes deeper into SELECT and the filtering techniques used in real applications and interview problems.

t; Core idea: SELECT decides which columns appear in the result. WHERE
t; decides which rows are allowed into the result. You can use them
t; independently or together.

10.1 Selecting All Columns and Specific Columns

The SELECT statement retrieves data from a table. Use * when every column is required; use specific column names when an application, report, or screen needs only selected information.

```
SELECT *  
FROM students;
```

The asterisk (*) means all columns from the table written after FROM.

```
SELECT student_name  
FROM students;
```

Only student_name is displayed. The other columns still exist in the table; they are simply not included in this result.

Selecting Multiple Columns

```
SELECT student_name, score  
FROM students;
```

Separate multiple column names with commas. Do not add a comma after the final column.

t; Column order matters: The result columns appear in the same order in
t; which they are written in SELECT.

```
SELECT score, student_name  
FROM students;
```

This displays score first and student_name second. It does not change the table structure.

Columns and Rows Are Different

Selecting two columns does not mean selecting two rows. SELECT controls the columns displayed; without WHERE, all rows are returned.

```
SELECT student_name, score  
FROM students;
```

t; When should you use SELECT *? It is useful for exploring a small
t; table or viewing the complete table. For application screens and
t; reports, selecting only required columns is usually clearer and avoids
t; retrieving unnecessary data.

10.2 Creating Column Aliases Using AS

A column alias is a temporary name given to a column in the query

result. It changes the heading displayed by the query, not the column name stored in the original table.

```
SELECT employee_name AS name
FROM employees;
```

The original column remains employee_name. Only the result heading becomes name.

Aliases for Multiple Columns

```
SELECT
    employee_name AS name,
    monthly_salary AS salary
FROM employees;
```

Each selected expression can have its own alias.

Aliases Containing Spaces

In MySQL, if an alias contains spaces, enclose it in backticks.

```
SELECT
    employee_name AS `Employee Name`,
    monthly_salary AS `Monthly Salary`
FROM employees;
```

t; AS is optional in MySQL: A query such as SELECT employee_name name
t; FROM employees; works, but using AS makes the intent clearer and is
t; preferred for readable learning and reporting queries.

Why Are Aliases Useful?

- Create readable report headings.
- Give meaningful names to calculated columns.
- Simplify long or technical column names in query results.
- Make application and reporting output easier to understand.

10.3 Creating Calculated Columns in Query Results

A calculated column is a value produced by an expression inside SELECT.

It does not have to exist as a stored column in the table.

```
SELECT
    product_name,
    price,
    quantity,
    price * quantity AS total_value
FROM products;
```

The expression price * quantity is evaluated separately for every row.

The calculation appears only in the query result; it does not modify the stored data.

Common Arithmetic Operators

Operator Operation Example

\+ Addition salary + bonus

\- Subtraction price - discount

* Multiplication price * quantity

/ Division total_marks / subjects

\% Remainder number % 2

Adding Values

```
SELECT
    employee_name,
    salary,
    bonus,
    salary + bonus AS total_pay
FROM employees;
```

Subtracting a Discount

```
SELECT
    product_name,
    price,
    discount,
    price - discount AS final_price
FROM products;
```

Calculating a Percentage

```
SELECT
    product_name,
    price,
    price * 18 / 100 AS tax_amount,
    price + (price * 18 / 100) AS final_amount
FROM products;
```

t; Parentheses: Use parentheses to make a calculation explicit and easier

t; to read, especially when an expression contains several operators.

Multiple Calculated Columns

```
SELECT
    product_name,
    price,
    price * 10 / 100 AS discount_amount,
    price - (price * 10 / 100) AS final_price
FROM products;
```

Order of Arithmetic Calculation

1. Parentheses

2. Multiplication and division

3. Addition and subtraction

```
SELECT
    price + price * 18 / 100 AS final_amount
FROM products;

SELECT
    price * (1 + 18 / 100) AS final_amount
FROM products;
```

Both expressions are intended to calculate the same 18% increase, while the parenthesized version makes the intended grouping especially clear.

t; Important: A calculated column in SELECT is not the same as UPDATE.

t; SELECT calculates a value for the result only; UPDATE changes stored

t; data.

10.4 Removing Duplicate Results Using DISTINCT

`DISTINCT` removes duplicate rows from the query result. It does **not** delete duplicate data from the table.

```
SELECT DISTINCT branch
FROM students;
```

This returns each branch only once.

DISTINCT with Multiple Columns

When multiple columns are selected, `DISTINCT` considers the **combination of all selected columns**.

```
SELECT DISTINCT branch, placement_status
FROM students;
```

Only duplicate combinations of `branch` and `placement\_status` are removed.

t; **Important:** `DISTINCT` changes the result of the query; it does not change the stored table data.

10.5 Sorting Results Using ORDER BY

`ORDER BY` sorts the rows returned by a query.

Sorting One Column

```
SELECT student_id, student_name
FROM students
ORDER BY student_id ASC;
```

`ASC` means ascending order and is the default:

```
SELECT student_id, student_name
FROM students
ORDER BY student_id;
```

Sorting in Descending Order

```
SELECT student_id, student_name, score
FROM students
ORDER BY score DESC;
```

`DESC` places larger values first for numeric data.

Sorting More Than One Column

```
SELECT student_id, student_name, branch, score
FROM students
ORDER BY branch ASC, score DESC;
```

The result is first sorted by `branch`. When two rows have the same branch, `score DESC` determines their order.

t; **Key idea:** `ORDER BY` changes the order of the query result. It does not permanently reorder the table.

10.6 Restricting Results Using LIMIT and OFFSET

`LIMIT` restricts the number of rows returned.

```
SELECT student_id, student_name
FROM students
LIMIT 3;
```

This returns only the first 3 rows of the result.

LIMIT with ORDER BY

For a predictable "top N" result, combine `ORDER BY` and `LIMIT`.

```
SELECT student_id, student_name, score
FROM students
ORDER BY score DESC
LIMIT 3;
```

This returns the 3 highest-scoring students.

OFFSET

`OFFSET` skips rows before returning the requested number.

```
SELECT student_id, student_name
FROM students
ORDER BY student_id
LIMIT 3 OFFSET 2;
```

This skips the first 2 rows and returns the next 3.

MySQL LIMIT Shortcut

MySQL also supports:

```
SELECT student_id, student_name
FROM students
ORDER BY student_id
LIMIT 2, 3;
```

Here:

- `2` = rows to skip

- `3` = rows to return

For learning and readability, `LIMIT 3 OFFSET 2` is often easier to understand.

LIMIT + OFFSET for Pagination

```
-- Page 1
SELECT *
FROM students
ORDER BY student_id
LIMIT 3 OFFSET 0;

-- Page 2
SELECT *
FROM students
ORDER BY student_id
LIMIT 3 OFFSET 3;

-- Page 3
SELECT *
FROM students
ORDER BY student_id
LIMIT 3 OFFSET 6;
```

t; **Important:** Use `ORDER BY` with pagination when you need a stable, predictable result order.

10.7 Filtering Records Using WHERE

The WHERE clause filters rows based on a condition. Only rows for which the condition is true are returned.

```
SELECT *
FROM students
WHERE branch = 'CSE';
```

The condition is checked for each row, and only CSE students appear in the result.

WHERE Syntax

```
SELECT column1, column2
FROM table_name
WHERE condition;
```

Filtering Text

```
SELECT *
FROM students
WHERE placement_status = 'Placed';
```

Text values must be enclosed in quotes.

Filtering Numeric Values

```
SELECT *
FROM students
WHERE student_id = 103;
```

Numeric values do not require quotes.

Selecting Columns and Filtering Rows Together

```
SELECT student_name, score
FROM students
WHERE branch = 'CSE';
```

SELECT controls which columns appear; WHERE controls which rows appear.

Correct Clause Order

```
SELECT student_name, placement_status
FROM students
WHERE branch = 'ECE';
```

t; Clause order matters: The normal form is SELECT ... FROM ... WHERE

t; Writing WHERE before FROM is incorrect SQL syntax.

10.8 Comparison Operators with Numbers, Text and Dates

Comparison operators compare a value with another value. A comparison produces a logical result: true when the condition matches and false when it does not.

Operator Meaning Example

= Equal to score = 75

\<\> Not equal to status \<\> 'Rejected'

\> Greater than score \> 75

\< Less than score \< 75

\>= Greater than or equal to score \>= 75

\<= Less than or equal to score \<= 75

Comparing Numbers

```
SELECT candidate_name, score
FROM job_applications
WHERE score > 80;

SELECT candidate_name, score
FROM job_applications
WHERE score < 75;
```

Understanding Boundary Values

Requirement Correct condition

Score is exactly 75 score = 75

Score is not 75 score \<\> 75

Score is above 75 score \> 75

Score is at least 75 score \>= 75

Score is below 75 score \< 75

Score is at most 75 score \<= 75

t; Boundary words: 'At least' and 'greater than or equal to' mean \>=.

t; 'At most' and 'less than or equal to' mean \<=. 'Above'/'greater than'

t; exclude the boundary; 'below'/'less than' exclude it.

Comparing Text

```
SELECT candidate_name, status
FROM job_applications
```

```

WHERE status = 'Shortlisted';

SELECT candidate_name, status
FROM job_applications
WHERE status <> 'Rejected';

```

Text comparisons depend on the stored value. For example, 'Under Review' is not the same stored text as 'UnderReview'.

Comparing Dates

Write dates using the YYYY-MM-DD format so MySQL can compare them chronologically.

```

SELECT candidate_name, applied_date
FROM job_applications
WHERE applied_date > '2026-08-15';

SELECT candidate_name, applied_date
FROM job_applications
WHERE applied_date >= '2026-08-15';

SELECT candidate_name, applied_date
FROM job_applications
WHERE applied_date < '2026-08-18';

```

The difference between > and >= is whether the boundary date itself is excluded or included.

10.9 Combining Conditions Using AND, OR and NOT

Real application requirements often contain more than one condition. SQL provides AND, OR, and NOT to combine or reverse conditions.

Operator Meaning Mental model

AND Every connected All requirements must
condition must be true match

OR At least one connected Any acceptable option
condition must be true can match

NOT The condition is The condition must be
reversed false

Using AND

Use AND when every condition must be satisfied.

```

SELECT student_name, branch, score
FROM students
WHERE branch = 'CSE'
AND score >= 80;

```

A row must satisfy both conditions: it must be CSE and have a score of at least 80.

Using More Than Two AND Conditions

```

SELECT student_name, branch, score, placement_status
FROM students
WHERE branch = 'CSE'
AND score >= 80
AND placement_status = 'Placed';

```

The row must satisfy all three conditions.

Using OR

Use OR when satisfying any one condition is enough.

```
SELECT student_name, branch
FROM students
WHERE branch = 'CSE'
OR branch = 'ISE';
```

This returns students from either branch. If one row satisfies both OR conditions, it still appears only once.

Using NOT

```
SELECT student_name, placement_status
FROM students
WHERE NOT placement_status = 'Placed';
```

This reverses the condition and returns rows whose placement_status is not 'Placed'. The same requirement can commonly be written as:

```
SELECT student_name, placement_status
FROM students
WHERE placement_status <> 'Placed';
```

t; Requirement words: 'and' usually indicates AND; 'either/or' usually

t; indicates OR; 'not', 'except', or 'excluding' usually indicates NOT.

t; Always translate the exact business requirement before writing the

t; SQL.

10.10 Controlling Condition Priority Using Parentheses

When AND and OR are mixed, SQL does not simply evaluate conditions from left to right. Parentheses explicitly group conditions and make the intended logic clear.

The relevant logical priority is:

1. *Parentheses*

2. *NOT*

3. *AND*

4. *OR*

Why Parentheses Matter

Suppose a training team wants students who belong to CSE or ISE and who have a score of at least 80.

```
-- Intended: (CSE or ISE) AND score >= 80
SELECT student_name, branch, score
FROM students
WHERE (branch = 'CSE' OR branch = 'ISE')
AND score >= 80;
```

Without the parentheses, the query can be interpreted according to operator precedence rather than the business grouping you intended.

```
-- Easy to misread:
SELECT student_name, branch, score
FROM students
WHERE branch = 'CSE'
OR branch = 'ISE'
AND score >= 80;
```

Because AND has higher priority than OR, the second query is treated

like:

```
branch = 'CSE'
OR (branch = 'ISE' AND score >= 80)
```

t; Best practice: Whenever AND and OR are mixed, use parentheses to show

t; the intended groups---even when the default precedence would happen to

t; produce the desired result.

Changing the Group Changes the Meaning

```
-- Group A: Development OR Testing, then salary requirement
SELECT *
FROM employees
WHERE (department = 'Development' OR department = 'Testing')
AND salary >= 50000;

-- Group B: every Development employee OR qualifying Testing employee
SELECT *
FROM employees
WHERE department = 'Development'
OR (department = 'Testing' AND salary >= 50000);
```

The conditions look similar, but the grouping produces different result

sets.

Parentheses with AND and OR Requirements

```
SELECT *
FROM students
WHERE branch = 'CSE'
AND (placement_status = 'Placed' OR score >= 90);
```

The student must be in CSE, and then at least one condition inside the

parentheses must be true.

10.11 Filtering from a List Using IN and NOT IN

IN checks whether a value matches any value in a specified list. It is a

cleaner alternative to writing many OR conditions for the same column.

```
SELECT student_name, branch
FROM students
WHERE branch IN ('CSE', 'ISE', 'ECE');
```

This is conceptually similar to:

```
SELECT student_name, branch
FROM students
WHERE branch = 'CSE'
OR branch = 'ISE'
OR branch = 'ECE';
```

NOT IN

```
SELECT student_name, branch
FROM students
WHERE branch NOT IN ('CSE', 'ISE');
```

This returns rows whose branch is not one of the listed values.

t; NULL caveat with NOT IN: If the tested column can contain NULL, NOT IN

t; can produce surprising results because comparisons with NULL are not

t; TRUE or FALSE in the ordinary sense. Use IS NULL/IS NOT NULL when you

t; specifically need to handle missing values.

10.12 Filtering Ranges Using BETWEEN

BETWEEN checks whether a value lies within an inclusive range. The boundary values are included.

```
SELECT student_name, score
FROM students
WHERE score BETWEEN 70 AND 80;
```

This includes scores of 70 and 80.

BETWEEN with Dates

```
SELECT candidate_name, applied_date
FROM job_applications
WHERE applied_date BETWEEN '2026-08-10' AND '2026-08-20';
```

For a DATE column, the start and end dates are included.

NOT BETWEEN

```
SELECT student_name, score
FROM students
WHERE score NOT BETWEEN 70 AND 80;
```

This returns values outside the inclusive range.

t; Interview point: BETWEEN is inclusive. If the requirement says

t; 'strictly greater than 70 and strictly less than 80', use score > 70

t; AND score < 80 instead.

10.13 Searching Patterns Using LIKE and Wildcards

LIKE performs pattern matching on text values. It is useful when the exact complete text is not known or when you want to search for a name, email, city, or other text pattern.

LIKE Wildcards

Wildcard Meaning Example

\% Matches zero or more 'A%' matches A, Asha,
characters Aarav

_ Matches exactly one '_a%' matches any
character string with a character
before a

Starts With

```
SELECT customer_name
FROM customers
WHERE customer_name LIKE 'A%';
```

Matches names beginning with A.

Ends With

```
SELECT customer_name, email
FROM customers
WHERE email LIKE '%@gmail.com';
```

Matches email values ending with @gmail.com.

Contains

```
SELECT customer_name
FROM customers
WHERE customer_name LIKE '%an%';
```

Matches names containing the sequence 'an'.

Exactly One Character at a Position

```
SELECT customer_name
FROM customers
WHERE customer_name LIKE '_a%';
```

The underscore represents exactly one character before a.

t; LIKE vs =: Use = when you want an exact comparison. Use LIKE when you

t; want pattern matching with wildcards.

10.14 Checking Missing Values Using IS NULL and IS NOT NULL

NULL represents a missing, unknown, or not-applicable value. NULL is not the same as 0, an empty string, or the text 'NULL'.

Checking for NULL

```
SELECT customer_name, email
FROM customers
WHERE email IS NULL;
```

This returns customers whose email value is NULL.

Checking for Non-NULL Values

```
SELECT customer_name, email
FROM customers
WHERE email IS NOT NULL;
```

This returns rows where email has a non-NULL value.

Why = NULL Does Not Work

```
-- Incorrect for checking NULL
SELECT *
FROM customers
WHERE email = NULL;

-- Correct
SELECT *
FROM customers
WHERE email IS NULL;
```

t; Remember: Use IS NULL and IS NOT NULL for NULL checks. Do not use =

t; NULL or \<\> NULL.

10.15 Putting the Filtering Operators Together

The real power of DQL comes from combining selection, calculated columns, aliases, and filtering in one query.

```
SELECT
    product_name AS `Product Name`,
    price AS `Unit Price`,
    quantity,
    price * quantity AS `Total Value`
FROM products
WHERE price BETWEEN 500 AND 5000
AND quantity >= 2;
```

This query demonstrates four ideas at once: selected columns, aliases, a calculated column, and a WHERE clause containing multiple conditions.

```
SELECT
    customer_name,
```

```

        email
FROM customers
WHERE (city IN ('Bengaluru', 'Hyderabad')
      OR email LIKE '%@gmail.com')
AND email IS NOT NULL;

```

Parentheses make the OR group explicit, IN handles a list, LIKE performs a pattern search, and IS NOT NULL removes missing emails from the final result.

10.16 Common DQL Mistakes

Mistake Why it is wrong Correct idea

SELECT * when only two Retrieves unnecessary Select only the columns are needed columns and makes required columns application output less precise

WHERE before FROM Incorrect SQL clause SELECT ... FROM ... order WHERE ...

Forgetting quotes Text values must be WHERE branch = 'CSE' around text represented as string literals

Using = NULL NULL requires special Use IS NULL handling

Using \> when Excludes the boundary Use \>= for 'at least' requirement says at least

Mixing AND and OR Can produce unintended Use parentheses without grouping logic

Using LIKE for an exact Pattern matching is Use = for exact match unnecessarily less direct equality

Forgetting that BETWEEN Boundary values are Use \>\< if boundaries is inclusive included must be excluded

10.17 SELECT vs WHERE --- Interview Mental Model

Question Answer

What does SELECT control? Which columns/expressions appear in the result.

What does WHERE control? Which rows satisfy the filter and appear in the result.

Does WHERE change the table? No. A SELECT ... WHERE query only reads and returns matching rows.

Does a calculated column change No. It exists only in that query stored data? result.

Does an alias rename the original No. It changes the result heading column? only.

10.18 Practical Course Tasks --- DQL Module

The screenshots you provided list the following practice/homework tasks.

They are preserved here as the module checklist.

Area Type Task

Selecting Data Practice Retrieve selected student information

Selecting Data Practice Create readable employee query results using aliases

Selecting Data Practice Calculate product values in a query

Selecting Data Homework Generate a formatted student report

Selecting Data Homework Generate a calculated order report

Filtering Data Practice Filter students based on their scores

Filtering Data Practice Filter employees based on department and salary

Filtering Data Practice Filter orders based on dates and amounts

Filtering Data Homework Retrieve products matching price conditions

Filtering Data Homework Retrieve job applications matching status conditions

Combining Conditions Practice Filter students using multiple conditions

Special Filtering Practice Filter students using

Operators IN and BETWEEN

Special Filtering Practice Search customer

Operators information using LIKE

Special Filtering Practice Find records containing

Operators missing values

Special Filtering Homework Build a product-search

Operators query

Special Filtering Homework Build a job-application

Operators filtering query

Worked Practice Examples

Practice 1 --- Filter Students by Score

```
SELECT student_name, score
FROM students
WHERE score >= 75;
```

Practice 2 --- Filter Employees by Department and Salary

```
SELECT employee_name, department, salary
FROM employees
WHERE department = 'Development'
AND salary >= 50000;
```

Practice 3 --- Filter Orders by Date and Amount

```
SELECT order_id, order_date, order_amount
FROM orders
WHERE order_date >= '2026-08-15'
AND order_amount > 3000;
```

Practice 4 --- Students Using IN and BETWEEN

```
SELECT student_name, branch, score
FROM students
WHERE branch IN ('CSE', 'ISE')
AND score BETWEEN 75 AND 95;
```

Practice 5 --- Customer Search Using LIKE

```
SELECT customer_name, email
FROM customers
WHERE customer_name LIKE 'A%';
```

Practice 6 --- Find Missing Values

```
SELECT customer_name, email
FROM customers
WHERE email IS NULL;
```

Interview Questions — DQL

Interview Questions --- DQL

Q1. What is the difference between SELECT and WHERE?

Ideal answer: SELECT chooses the columns or expressions displayed in the result; WHERE filters the rows that are allowed into the result.

Q2. Does SELECT ... WHERE modify the table?

Ideal answer: No. It only reads and returns matching data.

Q3. What does SELECT * mean?

Ideal answer: It selects all columns from the table specified in FROM.

Q4. How do you select multiple columns?

Ideal answer: Write the column names after SELECT and separate them with commas.

Q5. Does the order of columns in SELECT matter?

Ideal answer: Yes. It determines the order of columns in the query result.

Q6. What is a column alias?

Ideal answer: A temporary result heading assigned to a selected column or expression, commonly using AS.

Q7. Does an alias rename the original table column?

Ideal answer: No. It changes only the name displayed in that query result.

Q8. Can an alias contain spaces in MySQL?

Ideal answer: Yes. Enclose the alias in backticks, for example AS `Monthly Salary`.

Q9. What is a calculated column?

Ideal answer: A result column produced by evaluating an expression such as price * quantity inside SELECT.

Q10. Does a calculated column permanently store its result?

Ideal answer: No. Unless separately persisted through another operation, the calculation exists only in the query result.

Q11. What is WHERE used for?

Ideal answer: To filter rows according to a condition.

Q12. What is the difference between \> and \>=?

Ideal answer: \> excludes the boundary; \>= includes it.

Q13. What does BETWEEN do?

Ideal answer: It checks whether a value falls within an inclusive range.

Q14. Is BETWEEN inclusive?

Ideal answer: Yes. Both the lower and upper boundary values are included.

Q15. What is the difference between IN and OR?

Ideal answer: IN checks one expression against a list of values and is a

concise alternative to multiple equality comparisons joined by OR.

Q16. What is the difference between = and LIKE?

Ideal answer: = performs an equality comparison; LIKE performs pattern matching and supports wildcards.

Q17. What does % mean in LIKE?

Ideal answer: It matches zero or more characters.

Q18. What does _ mean in LIKE?

Ideal answer: It matches exactly one character.

Q19. Why can't you use = NULL to find missing values?

Ideal answer: NULL represents an unknown/missing value, so ordinary equality does not test for it. Use IS NULL or IS NOT NULL.

Q20. What is the difference between IS NULL and IS NOT NULL?

Ideal answer: IS NULL finds rows whose value is NULL; IS NOT NULL finds rows whose value is not NULL.

Q21. What does AND mean in a WHERE clause?

Ideal answer: Every connected condition must be true.

Q22. What does OR mean?

Ideal answer: At least one connected condition must be true.

Q23. What does NOT do?

Ideal answer: It reverses a condition.

Q24. What is the logical priority of parentheses, NOT, AND and OR?

Ideal answer: Parentheses have the highest grouping priority, followed by NOT, then AND, then OR.

Q25. Why are parentheses important when mixing AND and OR?

Ideal answer: They explicitly define which conditions belong together and prevent the query from being interpreted differently from the business requirement.

Q26. What is a common mistake with NOT IN and NULL?

Ideal answer: If the tested column contains NULL, NOT IN can behave unexpectedly. Handle NULL explicitly when it is possible.

Q27. Can WHERE be used with UPDATE and DELETE?

Ideal answer: Yes. WHERE can restrict which rows an UPDATE or DELETE affects, and it is critical for avoiding accidental changes to every row.

Q28. Can you use a SELECT alias in WHERE?

Ideal answer: In a normal query block, a SELECT-list alias generally cannot be referenced in WHERE because WHERE is evaluated before the SELECT list. Repeat the expression or use a subquery/CTE when appropriate.

Q29. What is the conceptual logical processing order of a simple SELECT query?

Ideal answer: FROM identifies the source rows, WHERE filters them, and SELECT produces the requested output expressions. This helps explain why SELECT aliases are generally unavailable in WHERE.

Q30. Why is selecting specific columns preferred in application queries?

Ideal answer: It makes the result clearer, avoids unnecessary data retrieval, and gives the application exactly the fields it needs.

Practice Tasks — DISTINCT, ORDER BY, LIMIT and OFFSET

****Practice 1 — Remove duplicate branches****

```
SELECT DISTINCT branch
FROM students;
```

****Practice 2 — Sort students by score****

```
SELECT student_id, student_name, score
FROM students
ORDER BY score DESC;
```

****Practice 3 — Find the top 3 scores****

```
SELECT student_id, student_name, score
FROM students
ORDER BY score DESC
LIMIT 3;
```

****Practice 4 — Pagination****

```
SELECT student_id, student_name
FROM students
ORDER BY student_id
LIMIT 3 OFFSET 3;
```

DISTINCT, ORDER BY, LIMIT and OFFSET — Interview Questions

****Q31. What does DISTINCT do?****

Ideal answer: `DISTINCT` removes duplicate rows from the query result. It does not delete or modify rows stored in the table.

****Q32. Does DISTINCT apply to multiple selected columns?****

Ideal answer: Yes. It considers the combination of the selected columns, so only duplicate combinations are removed.

****Q33. What is ORDER BY used for?****

Ideal answer: `ORDER BY` sorts query results by one or more columns.

****Q34. What is the difference between ASC and DESC?****

Ideal answer: `ASC` sorts in ascending order; `DESC` sorts in descending order. ASC is the default.

****Q35. Can ORDER BY sort by multiple columns?****

Ideal answer: Yes. The first column has the highest sorting priority, and later columns break ties.

****Q36. What does LIMIT do?****

Ideal answer: ``LIMIT`` restricts the number of rows returned by a query.

****Q37. What does OFFSET do?****

Ideal answer: ``OFFSET`` skips a specified number of rows before the query returns the rows requested by `LIMIT``.

****Q38. Why should ORDER BY often be used with LIMIT?****

Ideal answer: Without an explicit order, "first" rows are not a reliable way to express requirements such as the highest scores or newest records. `ORDER BY`` defines the required ordering before `LIMIT`` restricts the result.

****Q39. How are LIMIT and OFFSET useful in applications?****

Ideal answer: They can implement pagination by skipping rows from earlier pages and returning a fixed number of rows for the current page.

Chapter 10 — Final Takeaway

DQL is about asking precise questions of stored data. ``SELECT`` chooses what you want to see, ``DISTINCT`` removes duplicate results, ``ORDER BY`` sorts the result, ``LIMIT`` and ``OFFSET`` restrict how many rows are returned, aliases make output readable, calculated columns create values at query time, comparison operators express boundaries, ``AND`/`OR`/`NOT`` combine logic, parentheses control grouping, ``IN`` handles lists, ``BETWEEN`` handles inclusive ranges, ``LIKE`` handles text patterns, and ``IS NULL`/`IS NOT NULL`` handle missing values.

t; ****Golden rule for interviews:**** Translate the requirement into row conditions first, decide which columns must be displayed second, decide whether duplicates must be removed and how results should be ordered, then use ``LIMIT`/`OFFSET`` when the requirement restricts the result size.